KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Digital Address Book With Contact Categorization

Submitted by

SANJEEV G

25BIT143

Under the Guidance of

Mr Mugilan.R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Digital Address Book With Contact Categorization"** submitted in partial fulfillment of the requirements for the course **Java Programming** is a **record of my original work** carried out under the guidance of

Mr Mugilan R, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

(Sanjeev.G)

CERTIFICATE

This is to certify that the project entitled **"Digital Address Book With Contact Categorization"** is a **bona fide work** carried out by **SANJEEV G, B.Sc,Information Technology 'B' (2025-2028)** , **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Digital Address Book with Contact Categorization is a Java-based application designed to efficiently manage and organize personal and professional contact information. Traditional methods of storing contacts, such as handwritten address books or unstructured digital lists, often lead to difficulties in searching, updating, and categorizing contact details. This project aims to provide a structured and user-friendly system that simplifies contact management through digital automation.

The system allows users to add, edit, delete, and view contact details such as name, phone number, email address, and address. A key feature of this application is contact categorization, where users can organize contacts into groups such as family, friends, colleagues, or business. This categorization improves accessibility and enables quick retrieval of specific contacts.

The application is developed using Core Java concepts including classes, objects, arrays or collections, file handling, and exception handling. It can be implemented as a console-based application or enhanced with a graphical user interface using Java Swing. The system also provides search functionality to quickly locate contacts based on name or category.

The main objective of this project is to improve efficiency in managing contact information, reduce manual effort, and ensure better organization of data. The system generates a clear and structured view of stored contacts, making it easy for users to maintain and access their address book. This project demonstrates the practical application of object-oriented programming principles in Java and serves as a foundation for developing more advanced contact management systems in the future.

1. INTRODUCTION

In today's fast-paced digital world, managing personal and professional contacts efficiently has become increasingly important. Traditionally, many individuals maintain contact details using notebooks, diaries, or unorganized phone lists. This manual or semi-digital approach often leads to difficulties in searching for specific contacts, duplication of entries, lack of proper categorization, and the risk of losing important information.

With the advancement of computer technology, digital address book systems have become a practical solution for organizing and managing contact information effectively. A computerized system allows users to store, update, and retrieve contact details quickly while maintaining proper categorization such as family, friends, colleagues, or business contacts. This not only improves accessibility but also enhances productivity and communication.

The Digital Address Book with Contact Categorization is a Java-based application designed to efficiently manage and organize contact information. The system enables users to add, edit, delete, and search for contacts based on various attributes such as name, phone number, or category. It also allows categorization of contacts into predefined or custom groups, making it easier to filter and access specific sets of contacts when needed.

This project is developed using Core Java programming concepts such as classes, objects, methods, arrays or collections, loops, conditional statements, and exception handling. The application can be implemented as a console-based program or further enhanced with a graphical user interface (GUI) using Java Swing for a more interactive user experience.

The primary goal of this system is to simplify contact management, ensure better organization of data, and provide quick and reliable access to contact information. This project also demonstrates how Java can be used to build practical, user-friendly applications for everyday use.

1.1Objective

The main objective of the Digital Address Book with Contact Categorization is to develop a simple and efficient application using Core Java that automates the process of storing, organizing, and managing contact information.

The specific objectives of this project are:
•To design and develop a Java-based digital address book system.
• To store and manage contact details such as name, phone number, email, and address.
• To categorize contacts into groups such as family, friends, and professional contacts.
•To enable easy searching, updating, and deletion of contact records.
•To reduce manual effort and errors in maintaining contact information.
•To provide a user-friendly interface for efficient contact management.
• To demonstrate the implementation of Core Java programming concepts such as classes, objects, methods, loops, and conditional statements.
• To ensure quick and organized access to contact information whenever required.

1.2Overview

The Digital Address Book with Contact Categorization is a menu-driven Java application used to store and manage contact information efficiently.

The system allows users to add, update, delete, and search contacts based on details like name, phone number, and category. Contacts can be organized into groups such as family, friends, and professional for easy access.

It ensures quick retrieval and organized storage of data, and can be implemented as a console-based program or enhanced using Java Swing. This project shows how Java can be used to simplify contact management and improve efficiency.

1.3 Features

The Digital Address Book with Contact Categorization includes the following main features:

• Contact Management – Allows adding, updating, and deleting contact details such as name, phone number, email, and address.
• Contact Categorization – Organizes contacts into groups like family, friends, and professional.
• Contact Search – Enables quick search of contacts by name, phone number, or category.
• Data Storage – Maintains and stores contact information in an organized manner.
• Easy Access – Provides quick retrieval of contact details whenever required.
• User-Friendly Interface – Offers a simple and easy-to-use menu-driven interface.
• Update Functionality – Allows users to modify existing contact information easily.
• Efficient Management – Reduces manual effort and improves overall efficiency in handling contacts.

2.PROGRAM ANALYSIS

Program analysis is a key phase in software development where the problem is examined in detail and system requirements are identified. This step helps in understanding how the system should function and what features are needed to solve the problem effectively.

The analysis phase focuses on identifying issues in the existing manual system and designing a computerized solution that improves organization, efficiency, and accuracy

2.1 Problem Definition

In many households and offices, managing contacts is often done manually using notebooks, spreadsheets, or unorganized phone lists. This traditional method creates several challenges.

Some common problems include:
• Difficulty in searching for specific contacts
• Duplication of contact entries
• Lack of proper categorization of contacts
• Risk of losing important contact information
• Time-consuming updates and maintenance

Manual contact management increases the effort required to organize and retrieve information, which can lead to inefficiency and frustration.

Therefore, there is a need for an automated digital address book that can quickly store, categorize, update, and retrieve contact details.

The Digital Address Book with Contact Categorization is designed to solve these problems by providing a computerized solution that simplifies contact management and ensures accurate, organized, and easily accessible information**.**

2.2 Project Overview

The **Hotel & Restaurant Billing System** is a Java-based application developed to simplify the billing process in restaurants and hotels.

The system provides a menu-driven interface where users can select food items from a predefined list and specify the quantity of each item ordered by the customer. The system automatically calculates the total price of the selected items and generates a final bill.

The system includes the following functionalities:

• Displaying the restaurant menu with item prices
• Selecting food items and entering quantity

• Calculating item cost automatically
• Computing subtotal and tax amount
• Generating the final bill for the customer

This project is implemented using **Core Java programming concepts** and can be developed as either a **console-based program or a Java Swing GUI application**.

2.3 Scopes and Limitations

Scope

The scope of the **Hotel & Restaurant Billing System** includes the following:

• Managing restaurant menu items
• Processing customer food orders
• Calculating the total bill automatically
• Generating a detailed invoice for the customer
• Demonstrating the practical use of Java programming concepts

The system can be used in:

• Restaurants
• Hotels
• Cafeterias
• Food courts
• Small food businesses

3.SYSTEM DESIGN

System design is the phase where the structure and components of the system are planned before actual implementation. In this stage, the system is divided into modules, and the workflow of the application is defined. Proper system design ensures organized development and makes implementation easier.

The Digital Address Book with Contact Categorization is designed as a menu-driven application where users can add, update, delete, search, and categorize contacts efficiently.

The system is divided into several modules, with each module handling a specific function, such as managing contact details, categorizing contacts, searching contacts, and displaying organized contact information

3.1 Modules

The main modules of the Digital Address Book with Contact Categorization are explained below:

1. Contact Management Module
This module is responsible for adding, updating, and deleting contact details. Users can enter information such as name, phone number, email, and address.
Functions include:
• Adding new contacts
• Editing existing contact information
• Deleting unwanted contacts

2. Contact Categorization Module
This module allows users to organize contacts into groups such as family, friends, or professional.
Functions include:
• Assigning contacts to categories
• Viewing contacts by category
• Managing category information

3. Contact Search Module

This module enables users to quickly search for contacts based on various criteria.
Functions include:

• Searching by name, phone number, or email
• Filtering contacts by category

4. Display & Retrieval Module

This module displays contact details in an organized manner for easy access.
Functions include:
• Viewing all contacts
• Viewing contacts within a specific category
• Retrieving information for quick use

5. Exit Module

This module allows the user to safely exit the application after managing contacts. Selecting the exit option terminates the program.

4.METHODOLOGY

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Digital Address Book with Contact Categorization**, a structured development approach was followed to ensure the system meets the requirements and functions efficiently.

The development process includes the following stages:
• **Problem Analysis** – Understanding the issues in manual contact management and defining system requirements.
• **System Design** – Planning the structure, workflow, and modules of the application.
• **Program Development** – Writing Java code to implement the designed modules and functionalities.
• **Testing and Validation** – Verifying that the system works correctly and efficiently, and fixing any errors.
• **Result and Documentation** – Preparing final results, user guide, and project documentation.

4.1 Problem Analysis

In this phase, the problem is studied and analyzed in detail, and the requirements of the system are identified.

For the Digital Address Book with Contact Categorization, the main problems identified in manual contact management are:
• Searching for contacts manually is time-consuming.
• Risk of duplicate or missing contact entries.
• Difficult to organize contacts into meaningful groups.
• No automated system to quickly retrieve or update contact information.

Based on these problems, the system requirements were defined to develop an automated address book that simplifies contact management, ensures organized storage, and allows quick access to information.

4.2 System Design

In this stage, the overall structure of the system is planned before coding begins.

For the Digital Address Book with Contact Categorization, the system is divided into modules such as:

• Contact Management Module – For adding, updating, and deleting contact details.
• Contact Categorization Module – For organizing contacts into groups like family, friends, and professional.
• Contact Search Module – For quickly finding contacts by name, phone number, or category.
• Display & Retrieval Module – For viewing all contacts or contacts within a specific category.
• Exit Module – For safely closing the application.

The program flow and structure are designed to provide an easy-to-use, menu-driven interface so users can efficiently manage and access their contacts.

4.3. Program Development

After designing the system, the program is implemented using the Java programming language.

For the **Digital Address Book with Contact Categorization**, the following steps were followed during development:

• Creation of classes and methods for managing contacts and categories
• Implementation of a menu-driven interface for user interaction
• Writing code for adding, updating, deleting, and searching contacts
• Handling user input and validating data
• Displaying contact details in an organized format

Core Java concepts such as classes, objects, loops, conditional statements, arrays or collections, and methods were used during development.

4.4 Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

For the **Digital Address Book with Contact Categorization**, different test cases were used to check the functionality of the system.

Example Test Cases:

Test Case	Expected Result
Add new contact	Contact is added correctly
Update contact details	Changes are saved and displayed
Search contact	Correct contact information found
Delete contact	Contact is removed from the system
Invalid input	Error handled properly

This phase ensures that the program runs without errors and all features work accurately.

4.5 Result and Documentation

After successful testing, the results of the system are documented.

For the Digital Address Book with Contact Categorization, the results include successful addition, update, deletion, search, and categorization of contacts. The system demonstrates organized storage and quick retrieval of contact information.

The results are shown through program execution and screenshots displaying input and output for different operations. Proper documentation is prepared to explain the system's design, implementation, features, and overall working.

4.6 Algorithm

Algorithm for the Digital Address Book with Contact Categorization:

1. Start the program.
2. Display the main menu with options: Add, Update, Delete, Search, View Contacts, Exit.
3. Ask the user to select a menu option.
4. If Add Contact is selected:
    - Input contact details (name, phone number, email, address).
    - Assign contact to a category.
    - Save the contact in the system.
5. If Update Contact is selected:
    - Search for the contact by name or phone number.
    - Modify the desired details and save changes.
6. If Delete Contact is selected:
    - Search for the contact and remove it from the system.
7. If Search Contact is selected:
    - Input search criteria (name, phone number, or category).
    - Display matching contact(s).
8. If View Contacts is selected:
    - Display all contacts or contacts by category.

9. Repeat steps 3–8 until the user selects Exit.

10. End the program.

This algorithm ensures that all operations are handled systematically and the contacts are efficiently managed.

4.7 Flowchart

A flowchart visually represents the program flow and helps understand how the system works.

Flow for the Digital Address Book with Contact Categorization:

Start

↓

Display Main Menu

↓

Select Option (Add / Update / Delete / Search / View / Exit)

↓

Is Option Add Contact? → Yes → Input Contact Details → Assign Category → Save Contact → Back to Menu

↓

Is Option Update Contact? → Yes → Search Contact → Modify Details → Save Changes → Back to Menu

↓

Is Option Delete Contact? → Yes → Search Contact → Delete Contact → Back to Menu

↓

Is Option Search Contact? → Yes → Input Search Criteria → Display Matching Contacts → Back to Menu

↓

Is Option View Contacts? → Yes → Display All or Categorized Contacts → Back to Menu

↓

Is Option Exit? → Yes → End Program

5.SOURCE CODE

```
import java.util.ArrayList;
import java.util.Scanner;

// Contact class to store individual contact details
class Contact {
    String name;
    String phone;
    String email;
    String category;

    Contact(String name, String phone, String email, String category) {
        this.name = name;
        this.phone = phone;
        this.email = email;
        this.category = category;
    }

    @Override
    public String toString() {
        return "Name: " + name + ", Phone: " + phone + ", Email: " + email + ", Category: " +
category;
    }
}

// Main AddressBook class
```

```
public class DigitalAddressBook {
    static ArrayList<Contact> contacts = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\n--- Digital Address Book ---");
            System.out.println("1. Add Contact");
            System.out.println("2. Update Contact");
            System.out.println("3. Delete Contact");
            System.out.println("4. Search Contact");
            System.out.println("5. View All Contacts");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            sc.nextLine(); // Consume newline

            switch (choice) {
                case 1:
                    addContact();
                    break;
                case 2:
                    updateContact();
                    break;
                case 3:
                    deleteContact();
```

```
                break;
            case 4:
                searchContact();
                break;
            case 5:
                viewContacts();
                break;
            case 6:
                System.out.println("Exiting Address Book...");
                break;
            default:
                System.out.println("Invalid choice! Try again.");
        }
    } while (choice != 6);
}

// Method to add a new contact
static void addContact() {
    System.out.print("Enter Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Phone: ");
    String phone = sc.nextLine();
    System.out.print("Enter Email: ");
    String email = sc.nextLine();
    System.out.print("Enter Category (Family/Friend/Professional): ");
    String category = sc.nextLine();
```

```
    contacts.add(new Contact(name, phone, email, category));
    System.out.println("Contact added successfully!");
}

// Method to update an existing contact
static void updateContact() {
    System.out.print("Enter Name of contact to update: ");
    String name = sc.nextLine();
    for (Contact c : contacts) {
        if (c.name.equalsIgnoreCase(name)) {
            System.out.print("Enter New Phone: ");
            c.phone = sc.nextLine();
            System.out.print("Enter New Email: ");
            c.email = sc.nextLine();
            System.out.print("Enter New Category: ");
            c.category = sc.nextLine();
            System.out.println("Contact updated successfully!");
            return;
        }
    }
    System.out.println("Contact not found!");
}

// Method to delete a contact
static void deleteContact() {
    System.out.print("Enter Name of contact to delete: ");
    String name = sc.nextLine();
```

```
    for (int i = 0; i < contacts.size(); i++) {
        if (contacts.get(i).name.equalsIgnoreCase(name)) {
            contacts.remove(i);
            System.out.println("Contact deleted successfully!");
            return;
        }
    }
    System.out.println("Contact not found!");
}

// Method to search for contacts
static void searchContact() {
    System.out.print("Enter Name or Category to search: ");
    String keyword = sc.nextLine();
    boolean found = false;
    for (Contact c : contacts) {
        if (c.name.equalsIgnoreCase(keyword) || c.category.equalsIgnoreCase(keyword)) {
            System.out.println(c);
            found = true;
        }
    }
    if (!found) {
        System.out.println("No matching contacts found.");
    }
}

// Method to view all contacts
```

```
    static void viewContacts() {
        if (contacts.isEmpty()) {
            System.out.println("No contacts available.");
        } else {
            System.out.println("\n--- All Contacts ---");
            for (Contact c : contacts) {
                System.out.println(c);
            }
        }
    }
}
```

6.INPUT AND OUTPUT SCREENS

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

D:\java lab programs>javac DigitalAddressBook.java

D:\java lab programs>java DigitalAddressBook

--- Digital Address Book ---
1. Add Contact
2. Update Contact
3. Delete Contact
4. Search Contact
5. View All Contacts
6. Exit
Enter your choice: 1
Enter Name: sanjeev
Enter Phone: 3221365476
Enter Email: sanjeev09@gmail.com
Enter Category (Family/Friend/Professional): family
Contact added successfully!

--- Digital Address Book ---
1. Add Contact
2. Update Contact
3. Delete Contact
4. Search Contact
5. View All Contacts
6. Exit
Enter your choice: 1
Enter Name: rakshan
Enter Phone: 4568775154
Enter Email: 25bit143@gmail.com
Enter Category (Family/Friend/Professional): friend
Contact added successfully!
```

```
--- Digital Address Book ---
1. Add Contact
2. Update Contact
3. Delete Contact
4. Search Contact
5. View All Contacts
6. Exit
Enter your choice: 2
Enter Name of contact to update: sanjeev
Enter New Phone: 9889778965
Enter New Email: sanjuu@gmail.com
Enter New Category: professional
Contact updated successfully!

--- Digital Address Book ---
1. Add Contact
2. Update Contact
3. Delete Contact
4. Search Contact
5. View All Contacts
6. Exit
Enter your choice: 3
Enter Name of contact to delete: rakshan
Contact deleted successfully!
```

```
--- Digital Address Book ---
1. Add Contact
2. Update Contact
3. Delete Contact
4. Search Contact
5. View All Contacts
6. Exit
Enter your choice: 4
Enter Name or Category to search: sanjeev
Name: sanjeev, Phone: 9889778965, Email: sanjuu@gmail.com, Category: professional

--- Digital Address Book ---
1. Add Contact
2. Update Contact
3. Delete Contact
4. Search Contact
5. View All Contacts
6. Exit
Enter your choice: 5

--- All Contacts ---
Name: sanjeev, Phone: 9889778965, Email: sanjuu@gmail.com, Category: professional

--- Digital Address Book ---
1. Add Contact
2. Update Contact
3. Delete Contact
4. Search Contact
5. View All Contacts
6. Exit
Enter your choice: 6
Exiting Address Book...

D:\java lab programs>
```

7.DISCUSSION & CONCLUSION

7.1 Discussion

The Digital Address Book with Contact Categorization was developed to automate contact management for individuals and organizations. The system allows users to add, update, delete, search, and categorize contacts efficiently.

During development, Core Java programming concepts such as loops, conditional statements, methods, arrays/collections, and object-oriented principles were applied. The system successfully handles contact operations and maintains organized data.

The system was tested with various inputs to ensure that contacts are managed correctly, searches return accurate results, and categories are applied properly. The results show that the system reduces manual effort, prevents duplication, and improves efficiency in managing contact information.

7.2 Conclusion

The Digital Address Book with Contact Categorization demonstrates the practical application of Core Java programming concepts in developing a real-world software solution.

The system simplifies contact management, ensures accurate and organized storage of information, and allows quick access to contacts. This project helps students understand how Java can be used to create practical, menu-driven applications.

The system can be further enhanced by adding features such as:

- Database connectivity for permanent storage
- Graphical User Interface (GUI) for better usability
- Advanced search and filtering options
- Integration with email or messaging services for direct communication

8.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

1.https://www.oracle.com/java
2. https://www.geeksforgeeks.org
3. https://www.w3schools.com
4. https://www.javatpoint.com